arXiv:2607.17917v1 [cs.AI] 20 Jul 2026

PEARL: Auditable Repair for Scientific Reasoning Graph Extraction

Bohan Su[1,*], Pengze Li[2,*], Yuchen Lu[3], and Xi Chen[4,†]

[1] School of Computer Science, Wuhan University
bohansu@whu.edu.cn
[2] Artificial Intelligence Innovation and Incubation, Fudan University
lipz25@m.fudan.edu.cn
[3] Department of Computer Science, Faculty of Science, University of Bath
yl4002@bath.ac.uk
[4] College of Computer Science and Artificial Intelligence, Fudan University
x_chen@fudan.edu.cn
[*]Equal contribution. [†]Corresponding author.

Abstract. Scientific Reasoning Graph Extraction (SRGE) aims to recover explicit links among observations, evidence, intermediate claims, and paper-level conclusions. LLMs can produce graph-like scientific explanations, but their outputs often mix malformed syntax, drifting edge labels, incorrectly oriented roots, and weak source anchors. We propose PEARL (*Peircean Extraction via Abstraction and Repair Layer*), a training-free framework that turns noisy LLM graph responses into auditable reasoning graphs and repairs them toward strict semantic validity. PEARL first materializes explicit graph content under a closed Peircean schema, then uses matched evidence-grounded judge feedback to repair rejected edge types, local inference steps, and terminal roots while preserving an audit trail. On five 70-paper model archives from ARCHE, a benchmark for latent reasoning-chain extraction, PEARL raises strict gate passes from 0/350 for the LLM baseline to 300/350, with average REA improving from 0.339 to 0.906. The graphs provide a reliability layer for research-agent and AI scientist workflows that need inspectable reasoning traces rather than unconstrained graph regeneration. Code and audit artifacts are available at https://github.com/BohanSu/auditable-repair-reasoning-graphs/tree/300-350_workshop

Keywords: scientific reasoning graphs · LLM repair · research agents · graph validation · Peircean reasoning

1 Introduction

Scientific agents increasingly need to read papers, retain evidence, compare claims, and reuse intermediate reasoning in later hypothesis generation and experiment planning. For such research-agent workflows, a paper is not only a document to summarize. It is also a source of structured scientific reasoning. Scientific Reasoning Graph Extraction (SRGE) targets this structure by recovering

links among observations, cited evidence, intermediate claims, and paper-level conclusions [5,13]. Recent LLMs can emit graph-like explanations with semantically useful node text and edge labels [13,5,8,19], but a graph-like response is not automatically a reliable graph artifact. Raw responses can contain malformed DOT, extra prose, XML-like fragments, renamed edge labels, missing endpoints, multiple terminal roots, or roots oriented in the wrong direction.

This makes SRGE reliability a staged problem. Structured generation and constrained decoding reduce surface-format failures [11,2]. Scientific extraction and knowledge-graph systems populate structured fields or graph representations [6,8,19]. Verification benchmarks test whether claims are supported by evidence [4,16]. However, SRGE release forces all of these concerns into one artifact. The graph must be parseable, conform to a closed role taxonomy, preserve valid endpoints and terminal-root structure, and later satisfy semantic evaluation over entities and reasoning edges. If parsing, repair, and evaluation are conflated, a pipeline may either reject salvageable graph material or silently regenerate a graph that no longer reflects the original model output.

PEARL addresses this gap by treating raw LLM graph responses as conditional structural material for later repair rather than trusted graph artifacts. The pipeline first materializes explicit response content under a fixed source-referenced graph contract and converts paper evidence into a generation-final graph through LLM extraction plus autonomous structure repair. It then applies vote/filter materialization, rejected-unit repair, and claim-root construction under logged local edits. A fresh evaluation under a fixed strict gate determines whether the output is accepted as semantically valid or retained as a typed residual with metric and audit records. This staged flow is summarized in Fig. 1. The central boundary is simple. Structural recovery may normalize and repair graph form, but semantic repair must be tied to matched judge feedback and re-evaluated before acceptance.

PEARL makes three contributions. First, it defines a training-free SRGE materialization framework that converts heterogeneous LLM graph responses into auditable graph artifacts under a closed Peircean schema. Second, it adds judge-feedback repair for semantic correctness. Rejected local inference steps, edge types, and terminal roots are repaired through logged local operations and then judged again under the same CG/REA protocol. Third, it evaluates the framework on ARCHE across five LLM archives, showing that PEARL substantially improves strict semantic validity and releases graph outputs that can support later reasoning-data construction and AI scientist workflows.

2 Related Work

Scientific Reasoning Graphs. Scientific reasoning and graph-explanation tasks define schemas linking claims, evidence, and argumentative relations [13,5]. They clarify target graph content, but not release readiness for graph-like LLM responses. PEARL complements these tasks by asking when an emitted SRGE response can be safely repaired and admitted.

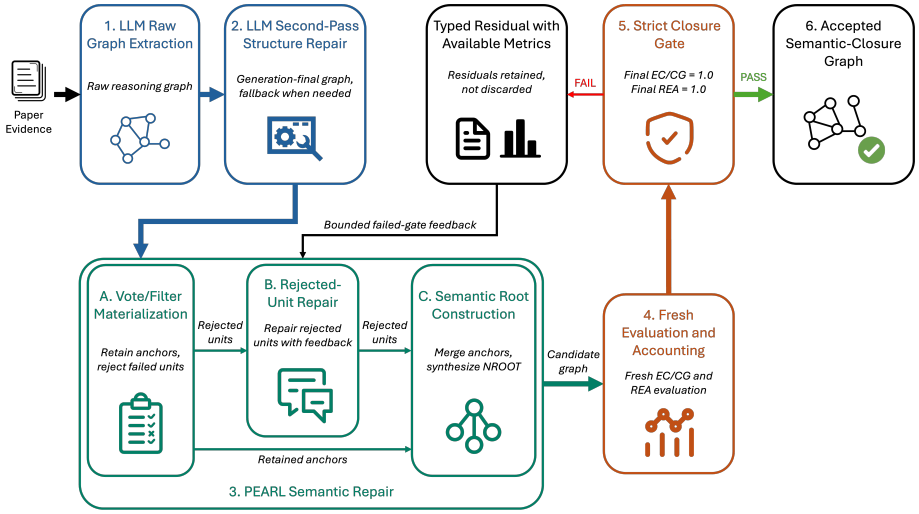


Fig. 1. Overview of the PEARL framework.

Structured Generation and Extraction. Constrained decoding, grammar-aware prompting, and schema checks reduce format errors in LLM outputs [11,2]. Scientific information extraction and graph-construction systems populate structured fields, knowledge graphs, or retrieval-oriented graph indices from scientific text [8,19,6]. These methods provide important upstream capabilities, but SRGE requires graph-level invariants over a single artifact, including closed Peircean labels, valid endpoints, terminal-root conventions, and source-reference fields. Parser compliance alone does not guarantee valid roles or root structure.

Self-Repair and Verification. Self-repair, tool-use, multi-agent feedback, and judge-based repair improve model outputs in many settings [14,17,20,7]. Scientific verification benchmarks separately evaluate whether generated claims are supported or contradicted by evidence [4,16]. PEARL combines these ideas with a stricter artifact boundary. Repair is not unconstrained whole-graph regeneration, but evaluator-guided correction over a matched source graph with final CG/REA re-evaluation. The accepted artifact must also remain evidence-linked after repair, not just locally plausible to evaluators.

3 Method

PEARL is a training-free, schema-first framework that turns noisy LLM graph outputs into auditable artifacts and, when possible, semantically valid graphs. Its core design decouples structural admissibility from semantic validity. The pipeline first stabilizes source-linked structure and then corrects reasoning errors using matched-judge feedback. This staged approach improves strict-gate passes without unconstrained whole-graph regeneration.

3.1 Source-Referenced Graph Contract

Given a paper and input fields, PEARL targets a directed reasoning graph

$$\mathcal{G} = (p, \mathcal{V}, \mathcal{E}, r), \tag{1}$$

where p identifies the paper instance, $\mathcal{V}$ is the set of graph nodes, $\mathcal{E}$ is the set of typed edges, and r is the terminal root of the released graph. Each node $v_i = (q_i, a_i, t_i)$ contains a local node identifier q_i, a source-reference tuple $a_i = (x_i, y_i, z_i)$, and natural-language text t_i. The integer coordinates identify a sentence (x_i), a viewpoint or cited reference within it (y_i), and an opinion within that reference (z_i); $(0, 0, 0)$ denotes implicit information. The source tuple is a structural pointer into the input paper fields rather than a proof of semantic entailment.

Each edge $e_j = (u_j, v_j, \tau_j)$ points from source node u_j to target node v_j and carries a Peircean role label [10]:

$$\tau_j \in \mathcal{T} = \{\text{ded-rule}, \text{ded-case}, \text{ind-common}, \text{ind-case}, \text{abd-phen}, \text{abd-know}\}. \tag{2}$$

Structural admissibility requires closed edge labels, known endpoints, no self-loops, and a single-root graph after structural repair. Target nodes must also have incoming support that can be assigned to a single Peircean reasoning family after normalization, meaning that the incoming roles for the target instantiate one coherent deduction, induction, or abduction pattern rather than mixing incompatible role families. We call one target node together with its valid paired incoming Peircean edges a reasoning unit. These are structural conditions. Semantic correctness is established only by later evaluator judgments.

3.2 Tolerance-Aware Structural Materialization

Stage 1 admits explicit structural material without inventing absent semantics. The pipeline extracts graph-like regions from the raw response and then attempts strict DOT parsing before invoking a syntax-tolerant recovery path. The tolerant path accepts DOT-like shapes, loose delimiters, trailing prose, and explicit XML/plist node-edge fields only when the corresponding nodes and edges are present in the response surface. It does not create missing scientific nodes, synthesize omitted edges, or complete partially specified claims.

The recovered output is normalized into a canonical graph representation, so downstream stages never depend on which parser succeeded. The parser branch remains only as audit metadata. From the normalized graph, PEARL derives a strict structural artifact for storage and an evaluation view for scoring. If single-root storage requires an engineering aggregation root, that root is treated as storage scaffolding and removed from the evaluation view, preventing storage constraints from being mistaken for scientific conclusions.

3.3 Deterministic Schema Repair

The recovered graph may still contain schema corruption, including edge-label aliases, invalid endpoints, self-loops, duplicate nodes, mixed-family targets, isolated nodes, or root-orientation errors. PEARL applies deterministic schema repair before semantic judgment. This stage canonicalizes edge-label aliases into the closed Peircean taxonomy, de-duplicates graph elements, removes invalid endpoints and self-loops, repairs root orientation under a fixed priority rule, and records every operation. When satisfying structural constraints would require unsupported inference, the pipeline records a typed failure instead of treating repair as semantic correction.

This repair is semantic-aware but not semantically unconstrained. For example, when an evaluator later rejects a reasoning unit because its Peircean edge type is wrong, PEARL can repair the edge/root configuration under that judgment. But such changes are routed through the judge-feedback stage and must be re-evaluated. They are not silently folded into structural parsing.

3.4 Judge-Feedback Semantic Repair

Stage 2 starts from the released source graph and its matched paper evidence under the same evaluation protocol. PEARL first runs a source evaluation to obtain content grounding (CG) and reasoning edge accuracy (REA). It also obtains local judge decisions over reasoning units, which separate majority-correct reasoning anchors from evaluator-rejected units. This matched evaluation is required because repair decisions may not consume stale votes from a different graph or from an earlier repair state.

PEARL then performs vote-guided repair. Majority-correct source reasoning is retained as the graph backbone, while rejected reasoning units are repaired under matched local judge feedback and reintroduced as repaired reasoning units. The method also recovers a claim root that expresses the paper-level research claim or proposal. Repair is therefore local and judge-conditioned. It targets rejected edge types, reasoning units, and roots rather than regenerating the entire graph from scratch.

The repaired graph is evaluated again under the same protocol. Previously accepted source reasoning may keep clean votes only when retained unchanged. Repaired reasoning, changed targets, and claim-root nodes must be judged again. If a repaired unit or root still fails, PEARL records the failure and may continue with bounded regeneration or pruning.

3.5 Final Gate and Provenance Audit

A candidate is strictly accepted only if it satisfies the unchanged final gate:

$$\mathrm{CG}_{\mathrm{final}} = 1.0 \quad \text{and} \quad \mathrm{REA}_{\mathrm{final}} = 1.0. \tag{3}$$

This gate defines semantic validity in the current framework. Non-accepted samples receive typed residual labels rather than being dropped. Some residuals occur before a terminal graph is produced, for example when there is no reliable

source anchor for repair. Others occur after final evaluation, for example when the repaired graph still misses the metric gate.

Provenance is audited separately from the strict final gate. For lexical source-span diagnostics, PEARL scores each node against its referenced paper span using the larger of two overlap scores, token F1 and node-to-span containment. At paper level, the audit reports whether most nodes have sufficient resolved source overlap and whether default-source nodes remain limited. These diagnostics do not change node text or edges and do not replace semantic evaluation. We also report MiniCheck [15] as an external source-grounding supplement. It checks graph-node claims against graph-identified source sentences, but it is not the strict acceptance gate.

4 Experiments

4.1 Experimental Setup

Data and Models. We evaluate on five model archives from ARCHE [5], a benchmark for latent reasoning-chain extraction: Claude Sonnet 4.5 [1], Gemini 3.1 Pro Preview [3], GPT-5.2 [9], Grok 4.1 Thinking [18], and Qwen3.5-397B-A17B [12]. Each archive contains 70 outputs under the SRGE prompt protocol, for $5 \times 70 = 350$ rows.

Baseline and Metrics. The LLM baseline is the generation-final graph after raw extraction and autonomous second-pass structure repair/fallback, before PEARL semantic repair. The primary metrics are CG and REA. The strict gate pass criterion is CG = 1.0 and REA = 1.0. MiniCheck source-grounding results are reported as diagnostics only.

4.2 Main Results

The five-model CG/REA results appear in Table 1. The LLM baseline does not pass the strict gate on any of the 350 rows, while PEARL passes on 300 rows. Average CG improves from 0.827 to 0.896, and average REA improves from 0.339 to 0.906. The larger REA gain shows that most of the remaining gap in the baseline is not entity mention coverage, but reasoning-edge and root correctness. This pattern matters for deployment because many baseline graphs are not empty or wholly unusable. They often retain enough local content to look plausible while still failing the global consistency conditions required for reliable reuse. In that failure regime, a repair layer is most valuable when it preserves anchored node content and targets edge/root correctness directly instead of replacing the entire graph with unconstrained regeneration.

Per-model behavior. PEARL improves REA for all five archives. GPT-5.2 reaches the strict gate on 68 of 70 rows and has final average REA of 1.000. The mixed-surface and open-weight archives are harder. Gemini, Grok, Claude, and Qwen

Table 1. Five-model SRGE results on ARCHE. Base is the LLM baseline before PEARL repair; PEARL is the terminal graph after judge-feedback repair.

Model	Strict Passes		CG		REA	
	Base	PEARL	Base	PEARL	Base	PEARL
Claude Sonnet 4.5 [1]	0/70	61/70	.785	.883	.219	.886
Gemini 3.1 Pro Preview [3]	0/70	53/70	.698	.806	.411	.829
GPT-5.2 [9]	0/70	68/70	.962	.992	.467	1.000
Grok 4.1 Thinking [18]	0/70	59/70	.823	.889	.277	.901
Qwen3.5-397B-A17B [12]	0/70	59/70	.869	.910	.319	.916
Aggregate	**0/350**	**300/350**	**.827**	**.896**	**.339**	**.906**

retain typed residuals, but each still moves from zero baseline strict passes to at least 53 passes. Across the 50 residual rows, the main typed causes are no reliable repair anchor (31 rows), final metric gate failure (13), metric regression (4), and final judge failure (2). The residual breakdown therefore identifies a fairly specific bottleneck. Most remaining failures arise before late-stage semantic acceptance, because the released source graph never provides a sufficiently trustworthy anchor for controlled repair. By contrast, metric regression and final-judge failure are comparatively rare, which suggests that matched re-evaluation prevents most harmful over-repair once a usable anchor is available.

The residual mix shows that archive-level gaps reflect both upstream recoverability and downstream repair. GPT-5.2 enters Stage 2 with cleaner source-linked reasoning units, whereas harder archives leave fragments with weak or incomplete source anchors. A lower pass count therefore need not indicate repair failure on an otherwise stable source graph. The residual taxonomy separates missing-anchor failures from graphs that reach re-evaluation but miss the metric gate under the same fixed evaluation protocol.

The hard rows also concentrate cost asymmetrically. When a reliable anchor is absent, extra evaluator rounds tend to revisit the same unresolved evidence boundary instead of producing new admissible structure. This pattern suggests that future efficiency gains are more likely to come from stronger source-link recovery before repair than from simply extending the number of semantic re-judgment cycles after repair has already stalled.

4.3 Diagnostic Ablation

We ablate the terminal graph used for final evaluation over the same five-model, 350-row scope. Raw extraction and the generation-final LLM graph nearly never pass the strict gate, with 1/350 and 0/350 passes. No root gen. omits semantic-root construction, Retained root keeps the original root without rejected-unit repair, and Root only performs root repair without iterative feedback. They reach 130/350, 125/350, and 102/350 passes, while full PEARL reaches 300/350. The pattern shows that deterministic retention alone leaves many graphs open and

root-only repair leaves evaluator-rejected local reasoning unresolved. Reliable paper-level closure depends on stable local units first, so a stronger root alone cannot compensate for reasoning edges that still fail re-evaluation. After local repair under matched judgments, the final claim root has a more stable support base. The gap between Root only and full PEARL further shows that local reasoning errors propagate upward, so claim-root repair cannot rescue a graph whose supporting edge families remain inconsistent.

4.4 Source-Grounding Supplement

MiniCheck evaluates whether graph-node claims are supported by the graph-identified source sentences. Across raw extraction, the LLM second-pass graph, and the PEARL terminal graph, the scoreable rows are 345/350, 350/350, and 317/350; support rates are .511, .584, and .557; and no-evidence rates are .267, .141, and .184. These numbers are a source-grounding supplement rather than a replacement for CG/REA, because the stages have different node sets and do not define one monotonic score. We use MiniCheck as an audit aid for source resolution quality, not as permission to bypass the stricter graph-level acceptance rule. Rows that fail the strict gate can still preserve locally grounded node text for audit, while rows that pass after repair may still deserve source-link review before downstream reuse. The drop from 350 to 317 scoreable terminal rows also shows that strict graph acceptance and node-level source checking fail for different reasons. This split gives downstream users a conservative signal for deciding when source links need manual audit. It also prevents local grounding from masking unresolved reasoning-edge errors.

5 Limitations

PEARL targets the ARCHE Peircean SRGE schema and five text-only archives. Portability to new schemas or multimodal evidence requires revised validators, frozen judge versions, and wider-deployment evidence checks.

6 Conclusion

PEARL provides auditable repair for LLM-generated scientific reasoning graphs under a strict CG and REA gate. Across five ARCHE archives, it raises strict gate passes from 0/350 to 300/350, showing that graph-like LLM outputs need audited repair before final acceptance in practical scientific extraction workflows.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Anthropic: Introducing Claude Sonnet 4.5. https://www.anthropic.com/news/claude-sonnet-4-5 (2025), accessed 2026-05-30
2. Geng, S., Cooper, H., Moskal, M., Jenkins, S., Berman, J., Ranchin, N., West, R., Horvitz, E., Nori, H.: Jsonschemabench: A rigorous benchmark of structured outputs for language models. arXiv preprint arXiv:2501.10868 (2025)
3. Google AI for Developers: Gemini 3.1 Pro Preview model documentation. https://ai.google.dev/gemini-api/docs/models/gemini-3.1-pro-preview (2026), accessed 2026-05-30
4. Javaji, S.R., Cao, Y., Li, H., Yu, Y., Muralidhar, N., Zhu, Z.: Can ai validate science? benchmarking llms for accurate scientific claim → evidence reasoning. arXiv preprint arXiv:2506.08235 (2025)
5. Li, P., Liu, J., Yu, J., Liu, L., Ding, M., Ouyang, W., Tang, S., Chen, X.: Arche: A novel task to evaluate llms on latent reasoning chain extraction. In: Proceedings of the AAAI Conference on Artificial Intelligence. vol. 40, pp. 1900–1908 (2026)
6. Li, S., Sadekar, A., Self, N., Su, Y., Andersland, L., Chaplin, M., Zhang, A., Yang, H., Henderson, J.B., Wigginton, K., et al.: Exploring llms for scientific information extraction using the sciex framework. arXiv preprint arXiv:2512.10004 (2025)
7. Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegreffe, S., Alon, U., Dziri, N., Prabhumoye, S., Yang, Y., et al.: Self-refine: Iterative refinement with self-feedback. Advances in neural information processing systems **36**, 46534–46594 (2023)
8. Mo, B., Yu, K., Kazdan, J., Mpala, P., Yu, L., Kanatsoulis, C., Koyejo, S.: Kggen: Extracting knowledge graphs from plain text with language models. Advances in Neural Information Processing Systems **38**, 30092–30115 (2026)
9. OpenAI: GPT-5.2 model documentation. https://platform.openai.com/docs/models/gpt-5.2 (2025), accessed 2026-05-30
10. Peirce, C.S.: Collected papers of charles sanders peirce, vol. 5. Harvard University Press (1934)
11. Poesia, G., Polozov, O., Le, V., Tiwari, A., Soares, G., Meek, C., Gulwani, S.: Synchromesh: Reliable code generation from pre-trained language models. arXiv preprint arXiv:2201.11227 (2022)
12. Qwen Team: Qwen3.5-397B-A17B model card. https://huggingface.co/Qwen/Qwen3.5-397B-A17B (2026), accessed 2026-05-30
13. Saha, S., Yadav, P., Bauer, L., Bansal, M.: Explagraphs: An explanation graph generation task for structured commonsense reasoning. In: Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing. pp. 7716–7740 (2021)
14. Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Hambro, E., Zettlemoyer, L., Cancedda, N., Scialom, T.: Toolformer: Language models can teach themselves to use tools. Advances in neural information processing systems **36**, 68539–68551 (2023)
15. Tang, L., Laban, P., Durrett, G.: Minicheck: Efficient fact-checking of llms on grounding documents. In: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing. pp. 8818–8847 (2024)
16. Wang, C., Shen, Y., Kuang, Z., Cohan, A., Zhao, Y.: Sciver: Evaluating foundation models for multimodal scientific claim verification. In: Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). pp. 8562–8579 (2025)

17. Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., et al.: Autogen: Enabling next-gen llm applications via multi-agent conversations. In: First conference on language modeling (2024)
18. xAI: Grok 4.1 model card. https://data.x.ai/2025-11-17-grok-4-1-model-card.pdf (2025), accessed 2026-05-30
19. Zhang, J., Qi, S., Dong, Y., Yuan, L., Shen, T., Fu, W., Wei, B., Zhu, H., Liu, J.: Gkg-llm: A unified framework for generalized knowledge graph construction. Information Fusion p. 103956 (2025)
20. Zheng, L., Chiang, W.L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E., et al.: Judging llm-as-a-judge with mt-bench and chatbot arena. Advances in neural information processing systems **36**, 46595–46623 (2023)